

PANTRYPAL: RECIPE HELPER CHATBOT

CM2015 Programming with Data - Midterm coursework report

PantryPal is a small recipe helper chatbot. The user can give ingredients, ask for a meal type such as breakfast, lunch or dinner, set a time limit and save a simple dietary preference during the current chat. The chatbot then recommends a recipe from a local recipe list. I chose this topic because it is practical, but not too close to the assignment process itself. It also gives enough scope to show regular expressions, preprocessing, dictionaries, lists, JSON data, a main loop and tests without making the project too large.

Application and use case

The chatbot is intended for a user who wants a quick meal idea from a small set of recipes. It is not a full recipe website and it does not try to give nutrition advice. Its main job is to understand short messages such as I have pasta and tomato, I am vegetarian, or I need dinner in 20 mins. From these messages it stores useful details and ranks the recipes that are available.

The external **intents.json** file stores the intent tags, regex patterns, response templates, ingredient aliases and recipe records. In the updated notebook it loads 15 intents, 19 regex patterns and 8 recipes. Keeping these details in JSON means the chatbot data can be edited without changing the main program logic.

Code organisation

I organised the notebook as a sequence of small functions rather than one long block of code. The main functions load the JSON file, build dictionaries for matching, clean and tokenise text, find the intent, update memory, score recipes, generate a response and run the chat loop. The main loop asks for input until the user types **exit** or **quit**. In the updated notebook I ran this loop manually and left the transcript visible, so re-running that cell will wait for keyboard input until one of those exit words is entered.

The main data structures are dictionaries and lists. The **pattern2intent** dictionary maps regex patterns to intent tags, while **intent2responses** maps each tag to possible response templates. A memory dictionary stores the name, diet, serving number, meal type, time limit and ingredients during one chat. The recipe list is a list of dictionaries, with each recipe storing its name, diet labels, meal labels, cooking time, ingredients and method. I also set **random.seed(13)** so the saved notebook output is reproducible while still showing that responses can be chosen from templates.

Pattern matching and preprocessing

The chatbot uses Python's **re** module for regular expression matching. Patterns are searched with **re.IGNORECASE**, so capitalisation does not change the result. Several patterns use named capture groups, for example **(?P<name>...)**, **(?P<diet>...)** and **(?P<minutes>...)**. These captured values are passed into the memory update step.

I added a simple preprocessing step for token inspection and ingredient matching. The **clean_text** function lowercases the input, changes curly apostrophes to plain apostrophes, removes punctuation and special characters used as noise, and collapses extra spaces. The **tokenise** function splits the cleaned text into words and numbers. I

then remove common stop words and create a stemmed version using the NLTK SnowballStemmer. I kept stemming as a supporting feature rather than using it to rewrite the whole message, because over-stemming can make results harder to understand.

Response generation and recipe scoring

Responses are selected from templates in the JSON file. Some templates contain placeholders such as **{name}**, **{diet}**, **{meal}** and **{ingredients}**, which are filled from the memory dictionary at runtime. The reset intent clears the memory dictionary and restores the starting values.

For recipe requests, PantryPal compares the current memory with each recipe. It gives extra score for matching ingredients, a matching meal type and fitting within the time limit. It also filters out recipes that conflict with the stored dietary preference. This makes the chatbot more flexible than a fixed one-question-one-answer design, but the logic is still simple enough to inspect and test.

Demonstration and tests

The updated notebook contains two demonstrations. The manual chat transcript shows the user asking for dinner ideas and then asking for a quick salad using cucumber, chickpeas, lemon and onion. The chatbot first suggests Egg fried rice for a broad dinner request, then correctly moves to Chickpea salad when the salad ingredients are given. The scripted demo uses Bob as the stored name, sets a vegetarian preference, then asks for pasta, tomato and garlic within 20 minutes; the chatbot recommends Tomato pasta and the memory summary keeps the name, diet, meal, time and ingredients. I also test the exit condition separately so that only exit or quit stop the main loop.

Test input	Expected behaviour	Why I included it
Hello!!! I've got pasta, tomato & garlic.	Cleaned to 'hello i ve got pasta tomato garlic', with pasta and garlic kept as useful tokens.	Checks punctuation, special characters and token handling.
My name is Bob	The set_name intent is matched and memory stores Bob.	Checks regex named groups and dynamic memory.
I have pasta, tomato and garlic and need dinner in 20 mins	The bot recommends Tomato pasta and stores dinner, 20 minutes and the ingredients.	Checks ingredient extraction, time matching, meal matching and recipe scoring.
I am vegan, then: Suggest lunch with chickpeas and tomato	The bot keeps the vegan preference and recommends Chickpea salad rather than Tuna wrap.	Checks memory across turns and dietary filtering.

Development reflection

In the first part of the module I focused on the basic Python development environment, variables, strings and simple input/output. The first chatbot version was just a small input and print loop. This helped me get the interaction working before I added more rules. In the next stage I added conditional logic, lists and Boolean checks, but I found that hard-coded words would not scale well as more intents were added.

I then moved the intent information into dictionaries and lists and split the program into functions. This made the notebook easier to test because each part had a clearer input and output. Moving the chatbot data into JSON was also useful, because patterns and responses no longer had to sit inside the main program code. In the final stage I spent most of the time on regular expressions, preprocessing and tests. The preprocessing feedback from the course was useful here, especially around punctuation, special characters, case sensitivity and avoiding aggressive stemming.

Limitations and further improvement

The project is intentionally small, so the recipe list and ingredient aliases are limited. Regex matching works well for predictable short messages, but it is less flexible with long or unusual sentences. The manual transcript also shows one useful limitation: a mixed message such as Hi my name is Bob can be treated as a greeting first because the program returns the first matching pattern. A later improvement would be to add intent priorities, so more specific matches such as names or time limits win over broader greeting patterns.

If I extended the project, I would add more recipes, more ingredient aliases and a clearer way to handle allergies. I would also move the tests into a separate test file once the notebook version is complete.

Conclusion

Overall, PantryPal demonstrates a functional chatbot built from the topics covered before the midterm. It uses an external JSON file, dictionaries and lists, regex pattern recognition, text preprocessing, memory-based response templates, a main loop, simple recipe ranking and assert-based tests. The final test cell in the updated notebook returns **All tests passed**, and the saved transcript shows the chatbot working across several turns.